

Chapter 5

Procedures

Procedures are an essential programming concept that support modularity, code reuse, and structured program design. In assembly language, procedures work closely with the stack and calling conventions to manage parameters, local variables, and return addresses.

5.1 Stack Operations

The **stack** is a special area of memory used for temporary storage of data, return addresses, parameters, and local variables. It follows the **Last In, First Out (LIFO)** principle.

Key Stack Registers

- **SP (Stack Pointer)** – points to the top of the stack
- **BP (Base Pointer)** – used to access parameters and local variables

Common Stack Instructions

Instruction Description

PUSH	Places data onto the stack
POP	Removes data from the stack
CALL	Pushes return address and jumps to procedure
RET	Pops return address and returns control

Example: Stack Operation

push ax

push bx

pop bx

pop ax

Importance of the Stack

- Supports procedure calls
- Preserves register values
- Enables nested and recursive procedures

5.2 Defining and Using Procedures

A **procedure** is a named block of code that performs a specific task and can be called from different parts of a program.

Procedure Structure

proc_name PROC

; procedure body

ret

proc_name ENDP

Calling a Procedure

call proc_name

Example: Simple Procedure

addTwo PROC

add ax, bx

ret

addTwo ENDP

Saving and Restoring Registers

Procedures should preserve register values using PUSH and POP instructions.

push ax

push bx

; procedure code

pop bx

pop ax

5.3 Linking to an External Library

External libraries contain reusable procedures that are linked to a program during assembly or linking time.

Advantages of Libraries

- Code reuse
- Reduced development time
- Standardized functionality

Linking Process

1. Declare external procedures using **EXTERN**
2. Link the object file with the library file

Example

EXTERN WriteString : PROC

Libraries allow assembly programs to use advanced input/output and system services.

5.4 The Irvine32 Library

The **Irvine32 library** is a widely used support library for learning assembly language using **MASM** on 32-bit Windows systems.

Features of Irvine32

- Simplified input/output operations
- String handling procedures
- File and console management

Common Irvine32 Procedures

Procedure	Description
-----------	-------------

WriteString	Displays a string
--------------------	-------------------

ReadInt	Reads an integer from keyboard
----------------	--------------------------------

WriteInt	Displays an integer
-----------------	---------------------

Crlf	New line
-------------	----------

Example Using Irvine32

```
include Irvine32.inc
```

```
.data
```

```
msg BYTE "Hello Assembly",0
```

```
.code
```

```
main PROC
```

```
mov edx, OFFSET msg
```

```
call WriteString
```

```
call Crlf
```

```
exit
```

```
main ENDP
```

```
END main
```

5.5 64-Bit Assembly Programming

64-bit assembly programming supports modern processors and operating systems. It uses extended registers and different calling conventions.

Key Features of 64-Bit Architecture

- 64-bit registers (RAX, RBX, RCX, RDX)
- Larger memory addressing
- Improved performance